Session & Cryptography

Architecture & Security Specification

Version 6.1

https://github.com/pixagram-blockchain/secure-api/

Pixagram SA, Zug, Switzerland — *Classification: Internal*

1. System overview

The Pixagram session system manages blockchain private keys (WIF format) in the browser. Keys must be available for signing transactions (votes, comments, transfers) but must be protected at rest and minimized in memory. The system provides two session types and an optional PIN layer.

Temporary sessions store keys in memory only. Closing the tab destroys them. **Persistent sessions** store keys in LacertaDB (IndexedDB), always encrypted with a device-bound AES-256-GCM CryptoKey. An optional PIN adds an inner Argon2id + ChaCha20-Poly1305 layer underneath the device encryption.

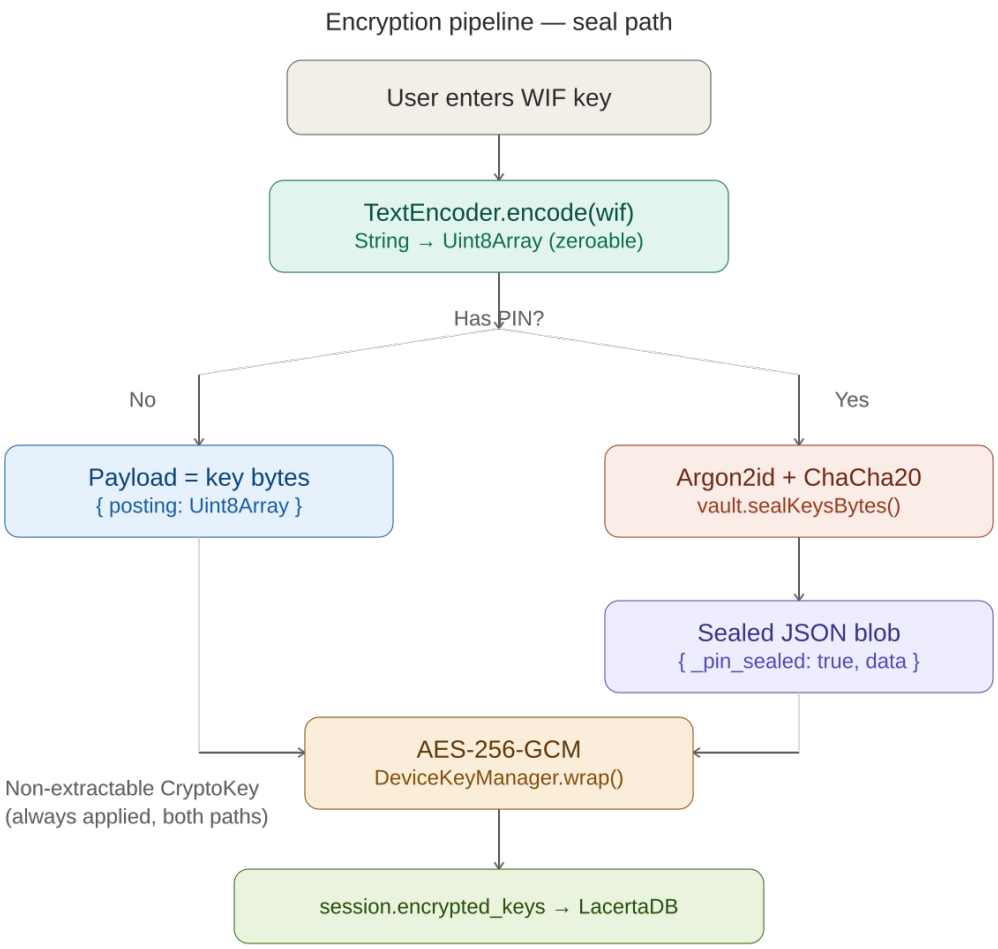
The session record is the single source of truth. Timeout values, Argon2 parameters, salt, and encrypted keys all live in one LacertaDB document per account. No external configuration lookups are needed to resume or unlock a session.

2. Encryption pipeline

When a user logs in, their WIF key string is immediately converted to a Uint8Array via TextEncoder.encode(). From this point, no string WIFs are stored in the session manager. The byte array is then encrypted through one or two layers depending on whether a PIN is set.

Without PIN: keys (Uint8Array) → AES-256-GCM(deviceKey) → LacertaDB. The device CryptoKey is non-extractable and stored in a separate raw IndexedDB via structured clone.

With PIN: keys (Uint8Array) → Argon2id(PIN, salt) → HKDF-SHA-512 → ChaCha20-Poly1305 → AES-256-GCM(deviceKey) → LacertaDB. The Argon2 parameters and salt are stored inside the session record so no external config is needed on resume.



2.1 Algorithm inventory

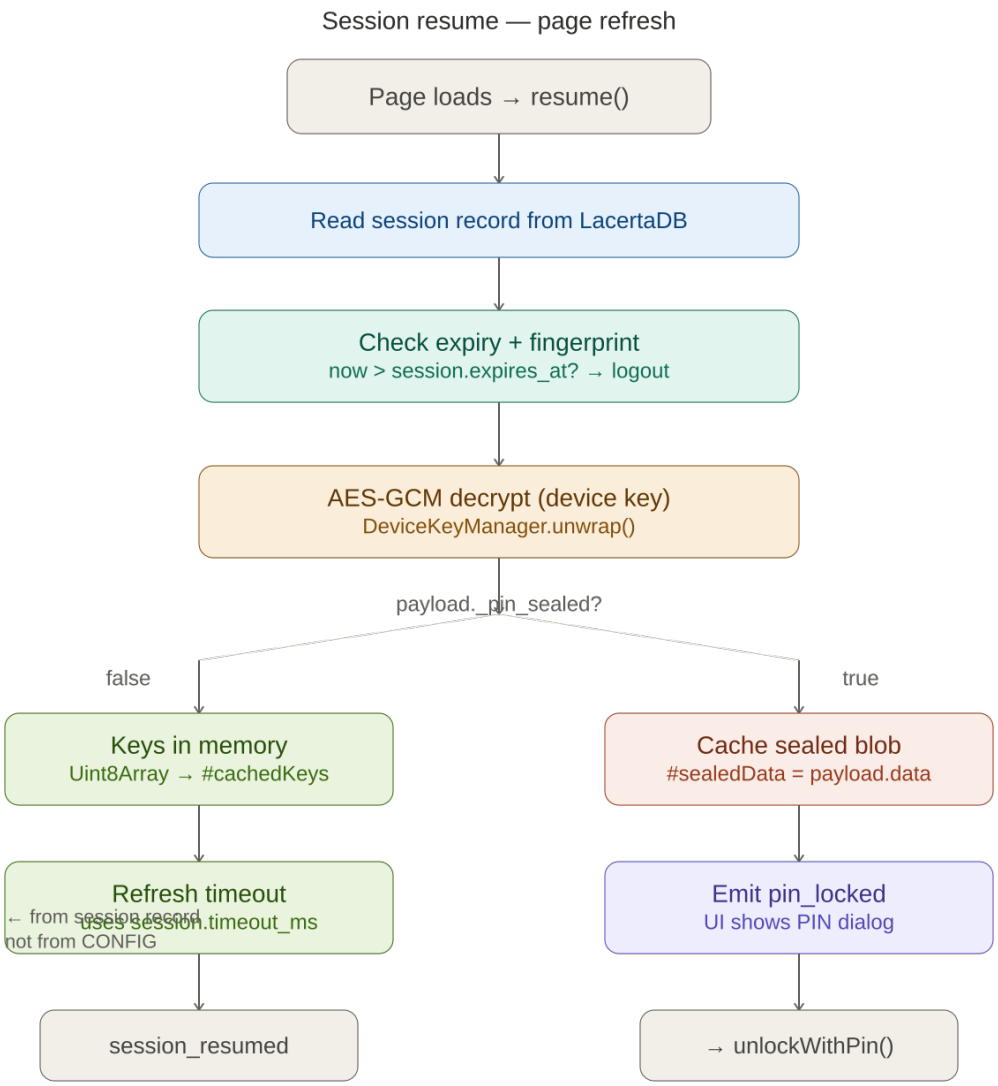
Algorithm	Purpose	Key size	Standard
Argon2id	PIN → master key	256-bit	RFC 9106
HKDF-SHA-512	Master → encrypt key	256-bit	RFC 5869
BLAKE3	Key commitment tag	256-bit	BLAKE3 spec
ChaCha20-Poly1305	Inner key encryption	256-bit key, 96-bit nonce	RFC 8439
AES-256-GCM	Outer device wrap	256-bit key, 96-bit IV	NIST SP 800-38D
SHA-256	Device fingerprint	256-bit	FIPS 180-4

3. Session resume flow

When the page loads, restoreSession() calls SessionManager.resume(). The flow reads the session record from LacertaDB, checks expiry (using the stored timeout_ms, not the runtime CONFIG default), verifies the device fingerprint, and device-unwraps the encrypted_keys blob.

If the payload contains _pin_sealed: true, the sealed blob is cached in memory (#sealedData) and a pin_locked event is emitted. The UI shows a PIN dialog. If no PIN, the decrypted key bytes go directly into #cachedKeys and the session is ready.

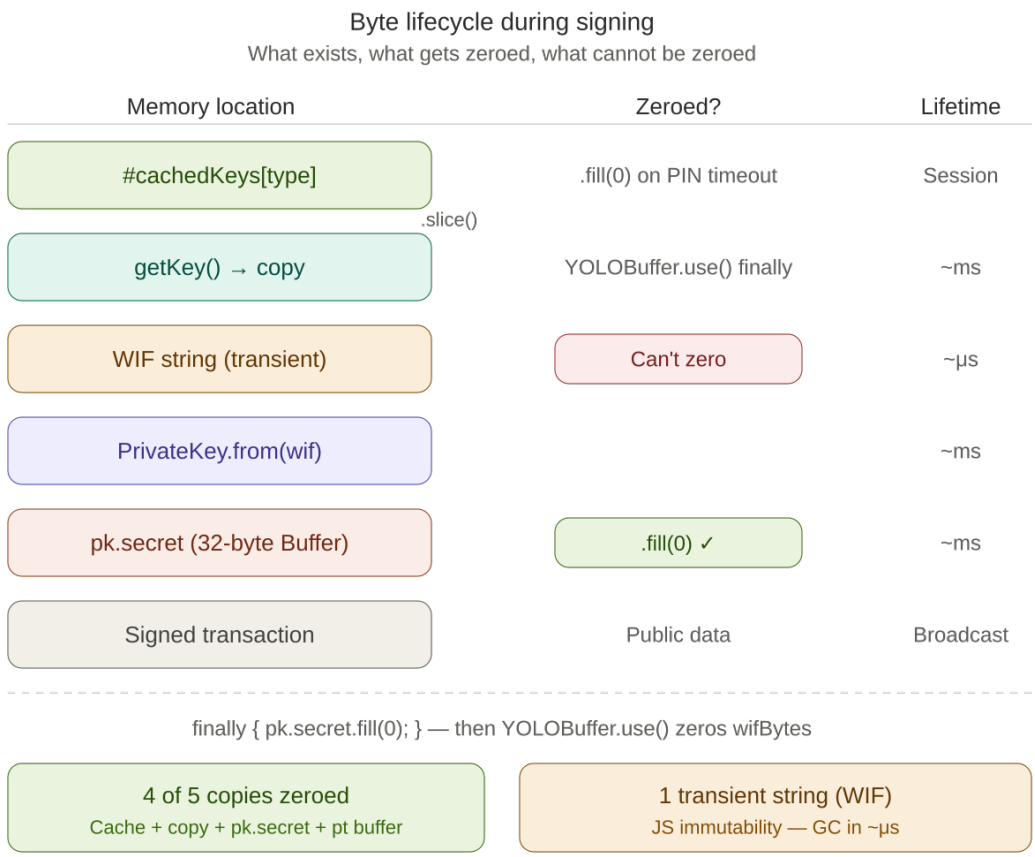
Critical fix (v6): The timeout refresh now reads session.timeout_ms from the session record, not this.config.SESSION_TIMEOUT from runtime CONFIG. This prevents page refresh from shortening a 12-hour session to the CONFIG default of 30 minutes.



4. Signing lifecycle & key zeroing

Keys are stored as Uint8Array (WIF UTF-8 bytes) in SessionManager.#cachedKeys. When a broadcast needs to sign a transaction, it calls getKeyAsYOLO(type) which returns a YOLOBuffer wrapping a copy of the key bytes. The copy is consumed inside YOLOBuffer.use() and automatically zeroed in the finally block.

Inside the signing callback, the WIF bytes are decoded to a string (transient, ~microseconds), passed to PrivateKey.from(wif), and the resulting PrivateKey object is used to sign. After signing, pk.secret.fill(0) zeros the 32-byte internal Buffer. The transient WIF string is the only copy that cannot be zeroed (JS string immutability), but it exists for microseconds and is immediately eligible for garbage collection.



4.1 Zeroing summary

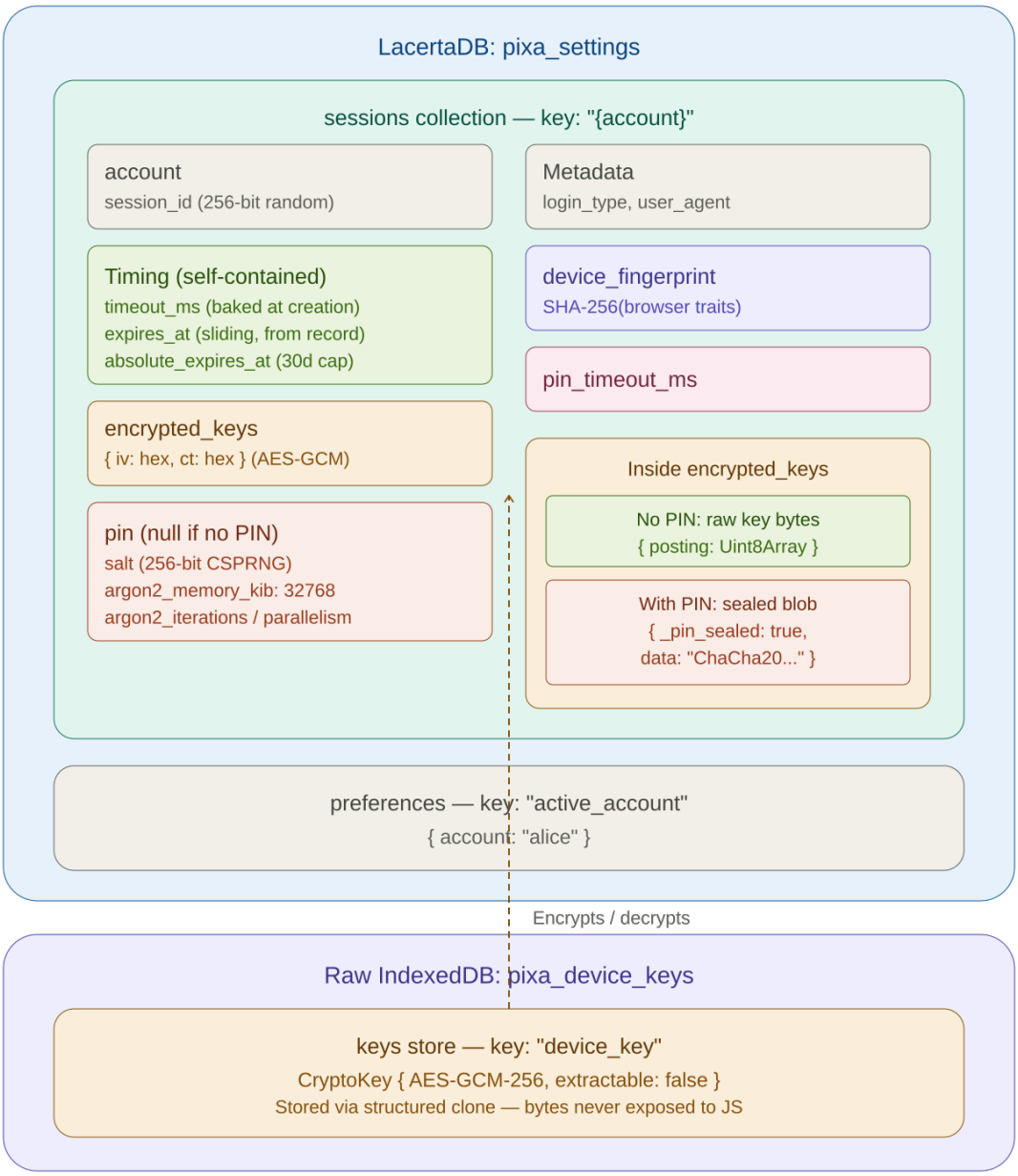
Location	Type	Zeroed ?	Method
#cachedKeys[type]	Uint8Array	Yes	.fill(0) on PIN timeout / logout
getKey() copy	Uint8Array	Yes	YOLOBuffer.use() finally block
WIF string	JS string	No	Immutable — GC only (~µs lifetime)
pk.secret	Buffer (32 bytes)	Yes	pk.secret.fill(0) in finally block

DeviceKeyManager pt	Uint8Array	Yes	.fill(0) after AES-GCM encrypt

5. Storage schema

The system uses two IndexedDB databases. pixa_settings (LacertaDB) stores session records and preferences. pixa_device_keys (raw IndexedDB) stores the non-extractable AES-256-GCM CryptoKey via structured clone. The CryptoKey is separate from LacertaDB because LacertaDB uses TurboSerial serialization which cannot handle CryptoKey objects.

Storage schema — single source of truth



5.1 Session record fields

Field	Type	Description
account	string	Normalized: lowercase, no @, trimmed
session_id	string	64 hex chars (32 random bytes via CSPRNG)
timeout_ms	number	Sliding window timeout, baked at creation. Never read from CONFIG.
expires_at	number	created_at + timeout_ms. Refreshed on resume/unlock.
absolute_expires_at	number	30-day hard cap from creation. Never refreshed.
pin_timeout_ms	number|null	PIN inactivity timeout. null if no PIN.
device_fingerprint	string|null	SHA-256 of browser characteristics.
encrypted_keys	{ iv, ct }	AES-GCM ciphertext (hex). Always device-wrapped.
pin	object|null	{ salt, argon2_memory_kib, argon2_iterations, argon2_parallelism }
login_type	string	"master", "posting", "active", etc.

6. Threat model

6.1 What the system protects against

Threat	Protection	Mechanism
IndexedDB file copy	Strong	Device CryptoKey non-extractable. PIN adds Argon2id brute-force barrier.
PIN brute-force	Strong	Argon2id 32 MiB + BLAKE3 key commitment. Lockout after 10, wipe after 50.
Session transplant	Moderate	Device fingerprint + CryptoKey binding.
Timing side-channel	Strong	Constant-time BLAKE3 commitment comparison.
Content injection / XSS	Strong	sanitize-html allowlists + SVG XSS analyzer (60+ event handlers).

6.2 Known limitations

Threat	Limitation	Mitigation
XSS (same-origin JS)	Attacker can use device CryptoKey	PIN layer adds Argon2id. CSP + SRI recommended.
Malicious extension	Full IndexedDB + JS access	Out of scope for client-side defense.
JS string immutability	WIF strings cannot be zeroed	YOLOBuffer minimizes string lifetime to ~µs.
Memory dump / cold boot	In-memory keys extractable	Standard limitation of all software wallets.

7. Argon2id parameters

Parameters are snapshotted into the session record at creation time and auto-tuned per device. The values below are defaults that exceed OWASP minimum recommendations.

Parameter	Default	OWASP minimum	Assessment
Memory	32 MiB	19 MiB	Exceeds minimum by 68%
Iterations	2	1	Exceeds minimum
Parallelism	3	1	p=3 → each attempt needs 96 MiB (3×32)
Output	256 bits	256 bits	Matches

Brute-force estimate: At 32 MiB / t=2 / p=3, a single Argon2id evaluation takes ~200ms. An 8-character PIN with mixed case, digits, and symbols (~52 bits entropy) requires ~2^{52} evaluations ≈ 142 million years on a single core. A pure-digit 8-char PIN (~26.5 bits) takes ~13 seconds on one core, which is why the MIN_PIN_ENTROPY check (30 bits) rejects such PINs.

8. Audit findings summary

Priority	Finding	Status
Critical	Session timeout override (30-min death)	Fixed in v6
Critical	PIN not prompted after refresh	Fixed in v6
Critical	Plaintext keys in LacertaDB	Fixed in v5/v6
High	Dual cold storage paths	Fixed in v6
High	R7 emergency memory scavenging	Fixed in v5
Medium	JS string immutability for WIFs	Mitigated (YOLOBuffer)
Low	Device fingerprint fragility	Open — advisory mode recommended
Low	Broadcast queue unencrypted	Open — low risk (public data)